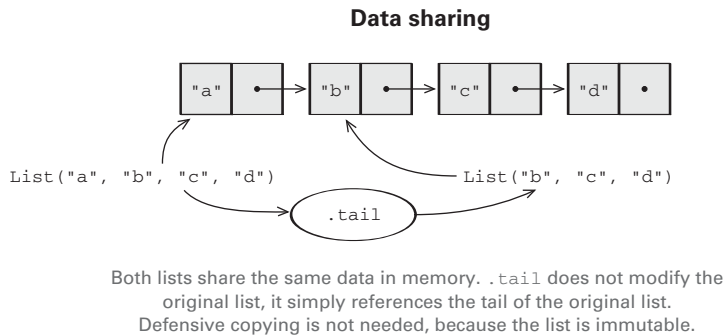


3.3 Data sharing in functional data structures

When data is immutable, how do we write functions that, for example, add or remove elements from a list? The answer is simple. When we add an element 1 to the front of an existing list, say `xs`, we return a new list, in this case `Cons(1, xs)`. Since lists are immutable, we don't need to actually copy `xs`; we can just reuse it. This is called *data sharing*. Sharing of immutable data often lets us implement functions more efficiently; we can always return immutable data structures without having to worry about subsequent code modifying our data. There's no need to pessimistically make copies to avoid modification or corruption.⁶

In the same way, to remove an element from the front of a list `mylist = Cons(x, xs)`, we simply return its tail, `xs`. There's no real removing going on. The original list, `mylist`, is still available, unharmed. We say that functional data structures are *persistent*, meaning that existing references are never changed by operations on the data structure.



Let's try implementing a few different functions for modifying lists in different ways. You can place this, and other functions that we write, inside the `List` companion object.



EXERCISE 3.2

Implement the function `tail` for removing the first element of a `List`. Note that the function takes constant time. What are different choices you could make in your implementation if the `List` is `Nil`? We'll return to this question in the next chapter.

⁶ Pessimistic copying can become a problem in large programs. When mutable data is passed through a chain of loosely coupled components, each component has to make its own copy of the data because other components might modify it. Immutable data is always safe to share, so we never have to make copies. We find that *in the large*, FP can often achieve greater efficiency than approaches that rely on side effects, due to much greater sharing of data and computation.